Control Flow

Control Flow is the order in which a computer executes instructions. By default, a program runs top-down, line by line. Control flow structures allow you to change that path, making the code skip or repeat based on specific conditions.

1. The if Statement

Evaluates a Boolean condition. If the condition is true, the code inside the curly brackets { } executes. If false, it skips that block entirely.

- **Syntax Note:** Unlike JavaScript or C++, Go does not require parentheses ( ) around the condition.

```
if age >= 18 {
   fmt.Println("You are an adult")
} else {
   fmt.Println("You are not an adult")
}
```

2. The for Loop (Go's Only Loop!)

In many languages, you have for, while, and do-while loops. **Go simplifies this by only having the for keyword.** You just write it in different ways to achieve different results:

- **The Standard Loop (3-Part):** Used when you know exactly how many times you want to iterate.

```
for i := 0; i < 10; i++ {
   fmt.Println(i)
}
```

- **The "While" Loop Equivalent:** Used when you just want to loop until a specific condition becomes false. You drop the initialization and update, leaving only the condition.

```
i := 0
for i < 10 {
   fmt.Println(i)
   i++
}
```

- **The Infinite Loop:** Used heavily in backend systems (like servers listening for requests). It loops forever until you force it to stop.

```
for {
   fmt.Println("Infinite loop")
}
```

3. for...range Loop

```
for _, value := range "helloworld" {
    fmt.Println(value)
}
```

1. The range Keyword:

Instead of manually setting up a count **(like i := 0; i < 10; i++),** you just use the keyword **range**. Go looks at the string, figures out how long it is, and loops through it one character at a time until it reaches the end.

2. The Two Return Values (Index and Value)

Whenever you use **range**, Go hands you back **two** pieces of information for every step of the loop:

1. The **Index** (the position, e.g., 0, 1, 2).
2. The **Value** (the actual data at that position, e.g., 'h', 'e', 'l').

3. The Blank Identifier (_):

Go is strictly compiled; if you declare a variable but never use it, the program will crash and refuse to build.

- If you wrote **for index, value := range...** but never actually used the index variable inside the loop, Go would throw an error.
- To fix this, Go gives you the **Blank Identifier** (the underscore _). It tells the compiler: I know range is handing me the index here, but I don't care about it. Throw it in the trash so I don't get an error.

NOTE: If you run this code, **it will NOT print the letters "h", "e".** It will print numbers like 104, 101! Why? Because range on a string pulls out the **runes** (the integer code points), not the text characters. To make it print the actual letters, you have to use the type conversion. **fmt.Println(string(value)).**

4. The switch Statement

When you have a long chain of **if / else if** checking the exact same variable, a switch statement is much cleaner to read.

- **How it works:** You provide a variable to the switch, and it checks it against a series of case values.
- **The default case:** If none of the specific cases match, the code inside the default runs.

```
i := 1
switch i {
case 1:
    fmt.Println("Case One")
case 2:
    fmt.Println("Case Two")
default:
    fmt.Println("Default Case")
}
```

NOTE: There is no **break** keyword for **switch** statements in Go.